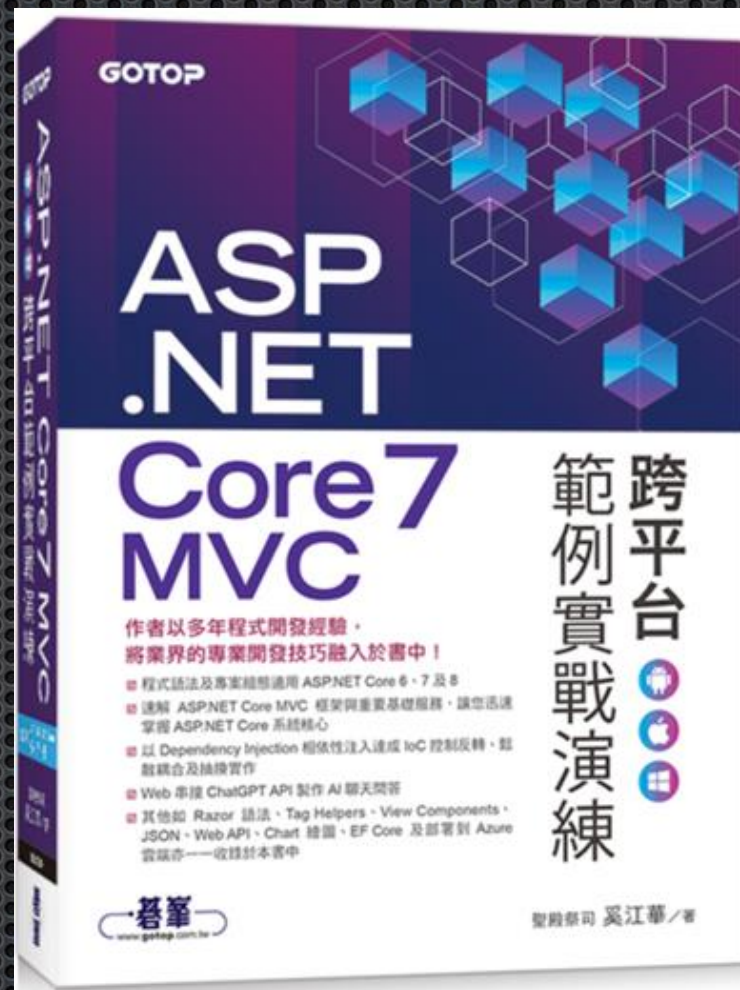


第10章

用Tag Helpers 標籤協助程式 設計Razor View檢視



作者：聖殿祭司。奚江華

大綱

10-1 Tag Helpers 標籤協助程式概觀

10-2 標籤協助程式之優點

10-3 Tag Helpers 與 HTML Helpers 的瑜亮情節

10-4 內建的標籤協助程式

10-5 Tag Helpers 加入、移除和範圍管理

10-6 自訂標籤協助程式

10-7 自訂標籤協助程式字型與色彩

Tag Helpers創造目的與定位

- Tag Helpers 標籤協助程式是用來擴充 HTML elements 屬性與功能，在語意上比 HTML Helpers 來得更直覺、清晰，修改與閱讀亦會更為容易，不致混淆或猜測
- 且因 ASP.NET Core Scaffolding 產出的 View 檢視，預設以 Tag Helpers 為主，HTML Helpers 為輔，使得熟悉 Tag Helpers 原理與使用方式，變成必要之務

Tag Helpers 標籤協助程式概觀

10-1

什麼是Tag Helpers標籤協助程式？

- 標籤協助程式是針對在 Razor檢視中 HTML elements 元素擴充出 **asp-for** 或 **asp-xxx**之類的屬性
- 而這些屬性可指定 C#程式，讓 Server 端程式可以參與 HTML 的建立與轉譯（Render）

```
<a asp-area="AreaBlogs"  
    asp-controller="Home"  
    asp-action="About">Area 的 Blogs/Home/About</a>
```

HTML 輸出：

```
<a href="/BlogZone/Home/About">Area 的 Blogs/Home/About</a>
```


原始 HTML 元素 + Tag Helpers 組合的好處

- 原始 HTML 元素使用上較為直覺，語意上也較為清楚
- 能讓 Server 端的 C# 語言及 .NET 物件參與 View 檢視設計，借用 Server 強大功能
- 透過簡單的語法，就能產出複雜的 HTML 結構

標籤協助程式之優點

10-2

標籤協助程式之優點

- HTML 友善的開發經驗
- 為 HTML 和 Razor markup 建立提供較為豐富的 IntelliSense 環境
- 使用 Server 端的 C#物件來產生更強健、易維護，且更具生產力的程式

Tag Helpers與HTML Helpers 瑜亮情節

10-3

Tag Helpers 與 HTML Helpers 的瑜亮情節

- 時序上，先發明 HTML Helpers，而後推出新的 Tag Helpers
- 而無論採用哪一種技術，最大的成本不在於「偏好使用哪一個」或「哪個比較優」，而是學習二者所必須投入的時間與精力成本變成雙倍
- Tag Helpers無法全面替代HTML Helpers，二者仍需相輔助，但以Tag Helpers為主，HTML Helpers為輔

Tag Helpers 與 HTML Helpers 的競合

- ① 主客易位
- ② 協同合作性
- ③ 功能替代性
- ④ Tag Helper 的學習/使用與否

1.主客易位

- ASP.NET Core世代，Tag Helpers 較HTML Helpers討好
- 從Scaffolding產出的CRUD檢視及_Layout樣板可見主客地位

```
<form asp-action="Create">
  <div asp-validation-summary="ModelOnly" class="text-danger"></div>
  <div class="form-group">
    <label asp-for="Name" class="control-label"></label>
    <input asp-for="Name" class="form-control" />
    <span asp-validation-for="Name" class="text-danger"></span>
  </div>
  <div class="form-group">
    <input type="submit" value="Create" class="btn btn-primary" />
  </div>
</form>
```


2. 協同合作性

- 因為 Tag Helpers 並未全面實作 HTML Helpers 所有功能，且 HTML Helpers 支援多載方法，但 Tag Helpers 沒有多載方法
- 如果你需要的功能 Tag Helpers 不支援，勢必還是得用 HTML Helpers 來做
- 以前 ASP.NET MVC 移轉到 ASP.NET Core，HTML Helpers 程式一樣可以運行，不需把所有的 HTML Helpers 改寫成 Tag Helpers
- 在 Tag Helpers 功能可發揮的地方，就以 Tag Helpers 為主；若 Tag Helpers 無法發揮的地方，就讓 HTML Helpers 上場

3. 功能替代性

- Tag Helpers 和 HTML Helpers 若功能有重疊到的部分，彼此就有替代性（alternative）產生，這是指可選擇 Tag Helpers 或 HTML Helpers 來完成

- Tag Helpers

```
<a asp-action="Edit" asp-route-id="@item.ProductId">Edit</a>  
<a asp-action="Details" asp-route-id="@item.ProductId">Details</a>  
<a asp-action="Delete" asp-route-id="@item.ProductId">Delete</a>
```

- HTML Helpers

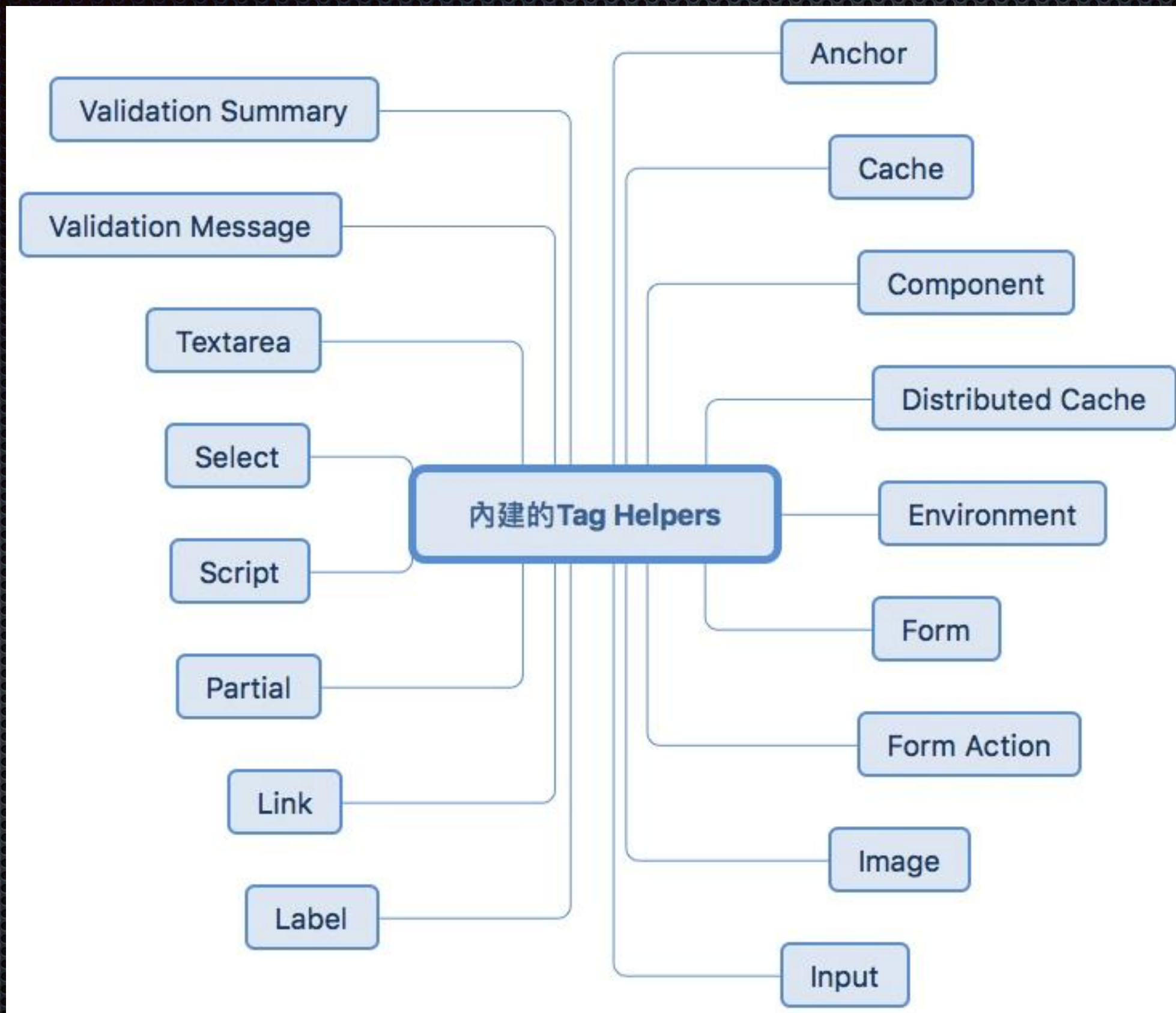
```
@Html.ActionLink("Edit", "Edit", new { id=item.Id })  
@Html.ActionLink("Details", "Details", new { id=item.Id })  
@Html.ActionLink("Delete", "Delete", new { id=item.Id })
```


4.Tag Helper 的學習/使用與否

- 若我很熟悉 HTML Helpers 了，是否可以不要學或用 Tag Helpers？就功能上來說，絕大多數可以辦到
- 但以團隊合作角度來說，別的開發者寫的 Code 是用 Tag Helpers，你是否有能力修改或維護？
- 倘若沒有，這是否產生了障礙，或顯示你的能力處於舊世代的尷尬場面？這值得個人去細細思量了

內建的標籤協助程式

10-4



<partial>部分檢視

10-4-1

<partial>部分檢視

- <partial>標籤協助程式是用來叫用Partial View部分檢視

<partial name="_PartialName" /> (呼叫 Partial View)

<partial name="_PartialName" view-data="ViewData" /> (用 view-data 屬性傳遞 ViewData)

<partial name="_PartialName" model="model 物件" /> (用 model 屬性傳遞 model 物件)

<partial name="_PartialName" for="model 物件" /> (用 for 屬性傳遞 model 物件)

<partial name="_PartialName" model="model 物件" view-data="ViewData" />

範例 10-1 用Partial Tag Helper呼叫部分檢視

呼叫_HeroPartial 部分檢視

```
<partial name="_HeroPartial" />
```

用 view-data 屬性傳遞 ViewData

```
<partial name="_HeroPartial" view-data="ViewData" />
```

用 model 屬性傳遞模型

```
<partial name="_HeroPartial" model="@Model" />
```

用 for 屬性傳遞模型

```
<partial name="_HeroPartial" for="@Model" />
```

同時傳遞模型和檢視

```
<partial name="_HeroPartial" model="@Model" view-data="ViewData" />
```


影像標籤

10-4-2

影像標籤

- Image 標籤協助程式是用來替加上版本號碼，且版本號碼具有唯一性
- 若 Server 端的靜態圖檔變更或修改，則會重新產生一組版本號碼，這會對靜態圖檔產生快取破壞的效果（cache-busting）
- 等於是強迫瀏覽器端重新從 Web Server 下載新版圖檔，而不能使用瀏覽器快取圖檔



Views/TagHelpers/ImageTagHelper.cshtml

加上版本號碼

```

```

HTML 輸出：

產生唯一的版號

```

```


<a>錨點

10-4-3

<a>錨點支援屬性

屬性	說明
asp-action	動作方法的名稱
asp-controller	控制器的名稱
asp-area	區域的名稱
asp-page	Razor 頁面的名稱
asp-page-handler	Razor 頁面處理常式的名稱
asp-route	路由名稱
asp-route-{value}	單一 URL 路由值，例如，asp-route-id="1234"
asp-all-route-data	所有路由值
asp-fragment	URL 片段
asp-protocol	URL 的通訊協定，例如 http 或 https
asp-host	Host 主機名稱

同時指定 asp-controller 和 asp-action 屬性值



Views/TagHelpers/AnchorTagHelper.cshtml

```
<a asp-controller="Product" ← 指定 Controller 名稱  
    asp-action="Index">Product/Index</a>  
    ↑  
    指定 Action 名稱
```

▪ HTML輸出

```
<a href="Product/Index">Product/Index</a>
```


<form>表單

10-4-4

<form>表單

- Form 標籤協助程式是用來產生 HTML 的<form>元素及其 action 屬性，設定方式有兩種：1.指定 Controller / Action 名稱，2.指定 Route 路由名稱
- 此外，還會自動產生隱藏的要求驗證權杖（Request Verification Token），而這在 HTML Helper 的 Html.BeginForm() 方法中，必須明確宣告 **@Html.AntiForgeryToken()** 才會產生

Form 標籤協助程式支援屬性

屬性	說明
asp-controller	控制器的名稱
asp-action	動作方法的名稱
asp-area	區域的名稱
asp-page	Razor 頁面的名稱
asp-page-handler	Razor 頁面處理常式的名稱
asp-route	路由名稱
asp-route-{value}	單一 URL 路由值。例如，asp-route-id="1234"
asp-all-route-data	所有路由值
asp-fragment	URL 片段

<form> 語法範例

```
<form asp-controller="Person" asp-action="PostData" method="post">
  <div class="form-group">
    <label asp-for="Id" class="control-label"></label>
    <input asp-for="Id" class="form-control" />
    <span asp-validation-for="Id" class="text-danger"></span>
  </div>
  <div class="form-group">
    <label asp-for="Name" class="control-label"></label>
    <input asp-for="Name" class="form-control" />
    <span asp-validation-for="Name" class="text-danger">
    </span>
  </div>
  <div class="form-group">
    <input type="submit" value="Submit"
      class="btn btn-primary" />
  </div>
</form>
```




範例 10-2 用 Form 標籤協助程式產生 action 屬性值

<Views/TagHelpers/FormTagHelper.cshtml>

Form Action 表單動作

10-4-5

Form Action 表單動作

- 一個 Form 表單中若有多個提交按鈕需指向不同的 URL，那麼 FormAction 可以很容易滿足此需求
- Form Action 標籤協助程式是針對 Form 表單內的 `<button>`、`<input type="submit">` 及 `<input type="image">` 產生 `formaction` 屬性，其屬性值是指向特定 URL
- `formaction` 屬性是 HTML5 功能，與前面 Form 標籤協助程式作用相同，但最大差異點是產生 URL 的層級不同

三種Form Action指定方式

```
<form method="post">
```

```
  <input asp-for="Id" placeholder="輸入你的 Id 號碼" /> <br /><br />
```

```
  <input asp-for="Name" placeholder="輸入你的名字" /> <br /><br />
```

1. 指定 Controller 與 Action

```
  <button asp-controller="Person" asp-action="PostData">提交資料</button> &emsp;
```

```
  <input type="submit" value="Submit" alt="" asp-controller="Person"  
    asp-action="QueryPersonalData" />
```

```
  <input type="image" src="~/images/SubmitButton.jpg" height="30" alt=""  
    asp-controller="Person" asp-action="QueryPersonalData" />
```

```
</form>
```




範例 10-3

用 Form Action 標籤協助程式產生 formaction 屬性值

<Views/TagHelpers/FormActionTagHelper.cshtml>

<label>標籤

10-4-6

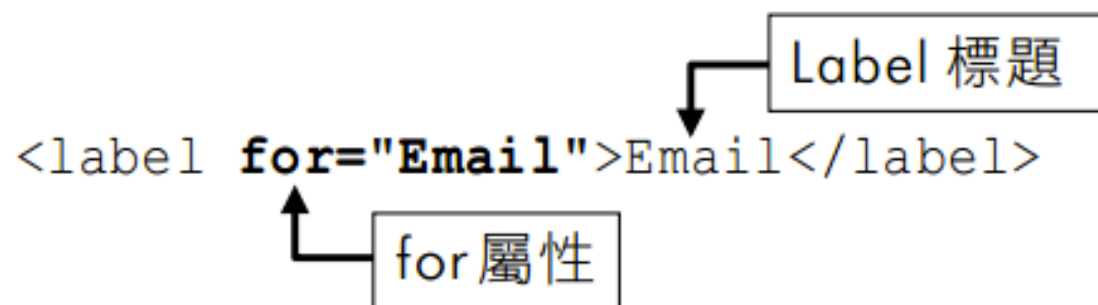
Label 標籤協助程式

- Label 標籤協助程式是用來產生<label>元素及其標題與for 屬性

 Views/TagHelpers/InputTagHelper.cshtml

```
@model RegisterViewModel
<form asp-controller="TagHelpers" asp-action="InputTagHelper" method="post">
    <label asp-for="Email"></label>
    <input asp-for="Email" />
    ...
</form>
```

HTML 輸出：



```
<label for="Email">Email</label>
```


<input> 輸入

10-4-7

<input>輸入

- 用來產生<input>元素，以及 id、name 及 type等屬性

```
public class RegisterViewModel
{
    public string Email { get; set; }
    public string Password { get; set; }
    public string ConfirmPassword { get; set; }
}
```

```
@model RegisterViewModel
<input asp-for="Email" />
<input asp-for="Password" />
<input asp-for="ConfirmPassword" />
```

```
<input type="email" id="Email" name="Email" value="" .../>
<input type="password" id="Password" name="Password" .../>
<input type="password" id="ConfirmPassword" name="ConfirmPassword" .../>
```


範例 10-4 使用 Label 及 Input 標籤協助程式 建立 Form 表單輸入畫面

電子郵件

密碼

確認密碼

Submit

```
public class RegisterViewModel
{
    [Required]
    [Display(Name = "電子郵件")]
    [DataType(DataType.EmailAddress)]
    4 references
    public string Email { get; set; }

    [Required]
    [Display(Name = "密碼")]
    [DataType(DataType.Password)]
    4 references
    public string Password { get; set; }

    [NotMapped]
    [Required]
    [Display(Name = "確認密碼")]
    [DataType(DataType.Password)]
    [Compare("Password", ErrorMessage = "密碼不符合!")]
    3 references
    public string ConfirmPassword { get; set; }
}
```


<select>選取

10-4-8

<select>選取

- 會根據 model 模型的屬性來產生<select>及<option>元素
- 它是 Html.DropDownListFor 和 Html.ListBoxFor的替代

```
<select asp-for="Country" asp-items="Model.Countries"></select>
```

HTML 輸出：

```
<select data-val="true" data-val-required="The Country field is required."  
id="Country" name="Country">  
<option value="US">USA</option>
```


範例 10-5 使用 Select 標籤協助程式 建立 Country 下拉式選單

TagHelper - CoreMvc3_Tag x +

localhost:44326/BuiltInTagHelpers/SelectTagHelper/

CoreTagHelpers Home 內建的Tag Helpers ▾

SelectTagHelper

Country

Submit

DisplayCountry - CoreMvc3_Tag x +

localhost:44326/BuiltIn

CoreTagHelpers

你選擇的國家為: USA

Back 返回上一頁

範例 10-6 Enum 列舉繫結到 Select 標籤協助程式

```
public enum CountryEnum
{
    [Display(Name = "美國")]
    USA = 10,
    [Display(Name = "日本")]
    Japan = 20,
    Canada = 30,
    France = 40,
    Germany = 50,
}
```

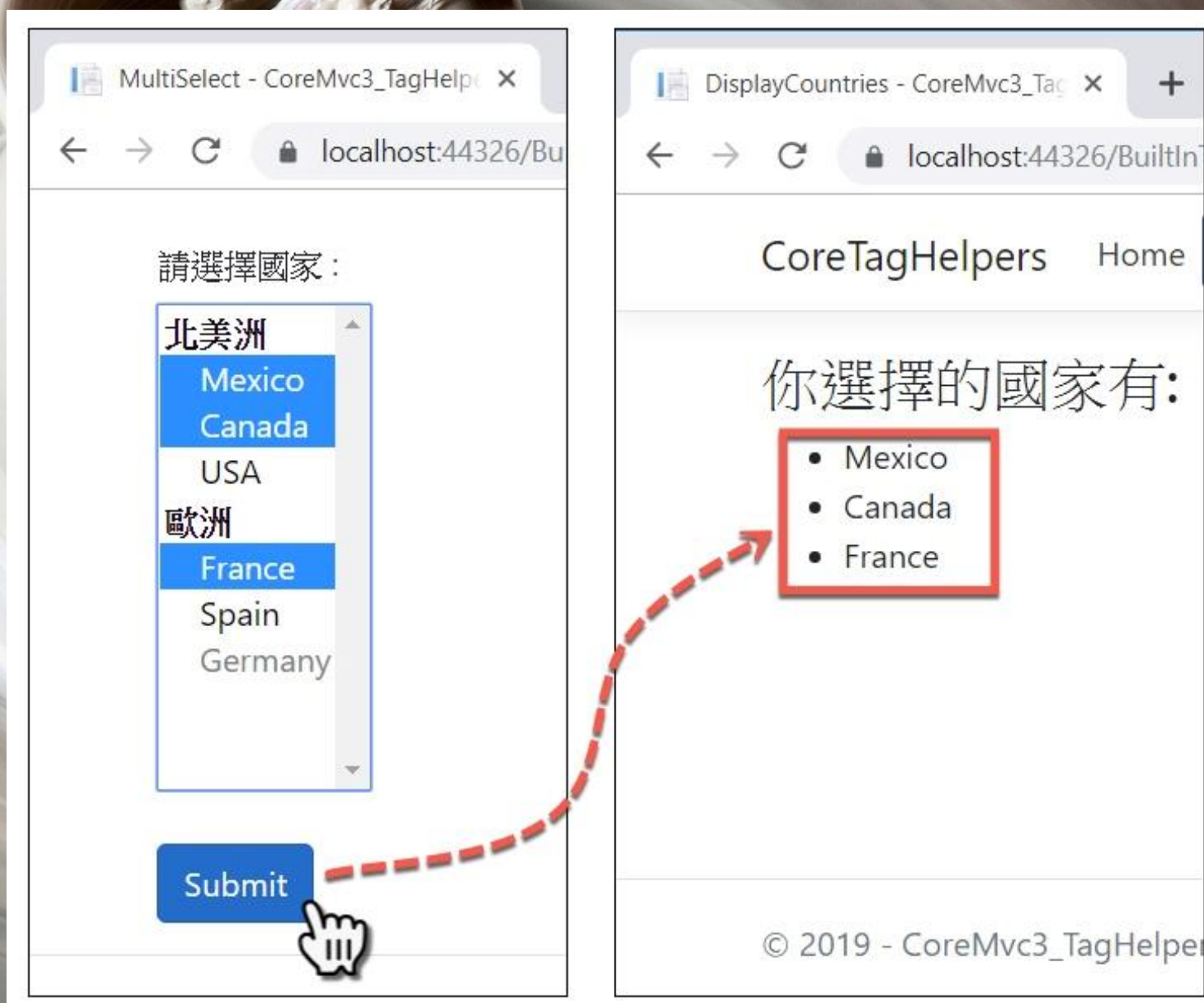
SelectEnum

==請選擇國家== ▼

美國
日本
Canada
France
Germany

Submit

範例 10-7 用Select標籤協助程式 產生具備<optgroup>選項群組的下拉選單



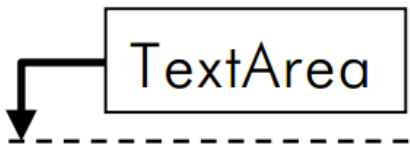
<textarea>文字區域

10-4-9

<textarea>

- TextArea 標籤協助程式會依據 asp-for 屬性產生 <textarea>元素及id、name 屬性
- 它是 Html.TextAreaFor 的替代

```
@model FeedbackViewModel
<form asp-controller="BuiltInTagHelpers" asp-action="TextareaTagHelper" method="post">
  <div class="form-group">
    <label asp-for="Opinion"></label>
    <textarea asp-for="Opinion" rows="4" cols="40"></textarea>
  </div>
  <button type="submit" class="btn btn-primary">送出</button>
</form>
```



The diagram illustrates the model binding process. A box labeled 'FeedbackViewModel' has an arrow pointing to the 'Opinion' property. This property is then linked to the 'asp-for="Opinion"' attribute in the <textarea> tag within the code snippet above.

範例 10-9 用 TextArea 標籤協助程式 輸入顧客意見調查



顧客Email

顧客意見

TextArea

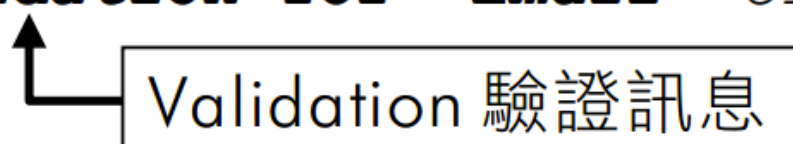
Validation 驗證訊息

10-4-10

Validation 驗證訊息

- 會根據 model 屬性產生驗證訊息。驗證不通過時，會產生驗證失敗訊息，藉以提醒輸入的資料格式不正確
- 它是 `Html.ValidationMessageFor()` 的替代

```
<input asp-for="Email" />  
<span asp-validation-for="Email" class="text-danger"></span>
```



Validation 驗證訊息

HTML 輸出：

```
<input type="email" d="Email" maxlength="50" name="Email" value="" .../>  
<span class="text-danger field-validation-valid"  
  data-valmsg-for="Email" data-valmsg-replace="true"></span>
```


在前端執行驗證

- 需在 View 加入以下程式

```
@section endJS
{
  <script src="~/lib/jquery-validation/dist/jquery.validate.js"></script>
  <script src="~/lib/jquery-validation-unobtrusive/jquery.validate.unobtrusive.min.js">
  </script>
}
```

- 或呼叫 `_ValidationScriptsPartial` 部分檢視

```
@section endJS
{
  <partial name="_ValidationScriptsPartial" />
}
```


Validation Summary 驗證摘要

10-4-11

Validation Summary 標籤協助程式

- 可彙總所有驗證失敗的驗證訊息，作摘要彙整顯示
- 在<div>的 asp-validation-summary 屬性設定顯示訊息的範圍

```
<div asp-validation-summary="ModelOnly" class="text-danger"></div>
```

驗證摘要的 HTML 輸出：

```
<div class="text-danger validation-summary-valid" data-valmsg-summary="true">  
  <ul>  
    <li style="display:none"></li>  
  </ul>  
</div>
```


ValidationSummary列舉值

- 三種列舉值，以控制驗證摘要訊息顯示範圍

表 10-3 ValidationSummary 列舉值

ValidationSummary 列舉值	說明
ValidationSummary.All	顯示所有類型的驗證錯誤訊息摘要，包括 Property 和 model 層級，以及自訂驗證錯誤訊息
ValidationSummary.ModelOnly	只顯示 model 屬性驗證錯誤訊息
ValidationSummary.None	不顯示驗證摘要訊息

範例 10-10 對 Input 輸入欄位做驗證訊息及驗證摘要



顧客Email The 顧客Email field is required.

顧客意見 The 顧客意見 field is required.

輸入的資料格式內容有誤!

<cache>快取

10-4-12

<cache>快取

- 可快取 ASP.NET Core 應用程式內容至快取提供者，以增加程式效能
- ASP.NET Core 支援數種快取，最簡單的是 IMemoryCache 記憶體快取
- IMemoryCache是將快取儲存在相同 Web Server 的記憶體之中，佔用相同主機的記憶體空間，同時也會受 Web Server 重新啟動而造成快取消失之影響
- IMemoryCache適合開發與測試場景使用

Cache 快取屬性

快取屬性	說明
enabled	啟用快取。可設為 true 或 false
expires-on	指定絕對過期日期
expires-after	於首次快取內容後多少時間過期
expires-sliding	設定快取項目值多久未被存取就會過期
vary-by-header	在標頭值改變時會觸發快取重新整理
vary-by-query	依查詢字串的改變，而觸發快取重新整理
vary-by-cookie	依 cookie 的改變，而觸發快取重新整理
vary-by-user	依@User.Identity.Names 改變，而觸發快取重新整理。可為 true 或 false
vary-by	當屬性字串值所參考的物件變更，而觸發快取重新整理
priority	設定快取保留的優先權層級，有 High、Normal（預設）、Low、NeverRemove 四種設定

<div>1. <cache>快取 (快取過期預設為 20 分鐘)</div>

<cache>

目前時間: @DateTime.Now

</cache>

<div>2. enable 啟用</div>

<cache **enabled="false"**> ← 不啟用快取

目前時間: @DateTime.Now

</cache>

<div>3. expire-on 指定絕對過期日期</div>

指定快取於 2025/3/28 日過期

<cache **expires-on="@new DateTime(2025,3,28,12,30,0)"**>

目前時間: @DateTime.Now

</cache>

<div>4. expire-after 於首次快取內容後多少時間過期 (預設為 20 分鐘)</div>

<cache **expires-after="@TimeSpan.FromSeconds(120)"**>

目前時間: @DateTime.Now

指定快取於 120 秒後過期

</cache>

<div>5. expire-sliding 設定快取項目的值多久未被存取就會過期</div>

<cache **expires-sliding="@TimeSpan.FromSeconds(60)"**>

目前時間: @DateTime.Now

60 秒未存取快取就過期

</cache>

範例 10-11 利用 Cache 標籤協助程式
設定網頁內容快取

<distributed-cache>分散式快取

10-4-13

<distributed-cache>

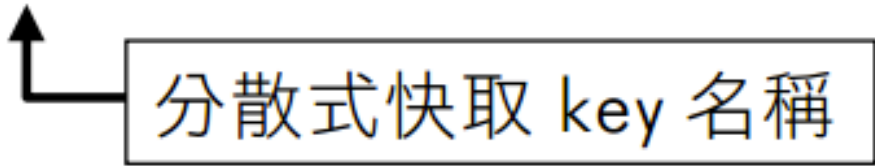
- 分散式快取可將內容快取至分散式快取來源，例如 SQL Server 或 Redis，藉以大幅提升 ASP.NET Core 應用程式效能
- 通常分散式快取服務是獨立於 Web Server 網頁伺服器之外，讓多台網頁伺服器共享相同的快取內容

分散式快取相較於本機 IMemoryCache之優點

- 分散式快取相較於本機 IMemoryCache 快取之優點
- 分散式快取相較於本機 IMemoryCache 快取之優點
- 不佔用網頁伺服器本機記憶體

分散式快取語法

```
<div class="alert alert-success">分散式快取以 name 作為快取的 key</div>  
<distributed-cache name="product-cache-unique-key-topsales">  
    目前時間: @DateTime.Now  
</distributed-cache>
```



分散式快取 key 名稱


```
//Distributed Memory Cache
services.AddDistributedMemoryCache();

//Distributed SQL Server Cache
services.AddDistributedSqlServerCache(options =>
{
    options.ConnectionString =
        _config["DistCache_ConnectionString"];
    options.SchemaName = "dbo";
    options.TableName = "TestCache";
});

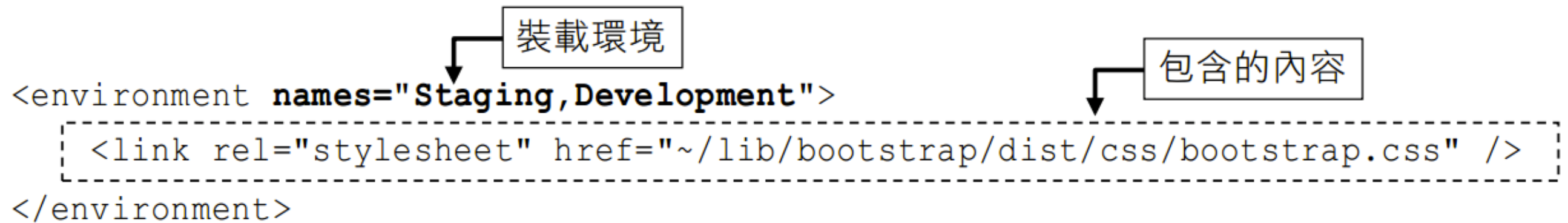
//Distributed Redis Cache
services.AddStackExchangeRedisCache(options =>
{
    options.Configuration = "localhost";
    options.InstanceName = "SampleInstance";
});
```


<environment>環境

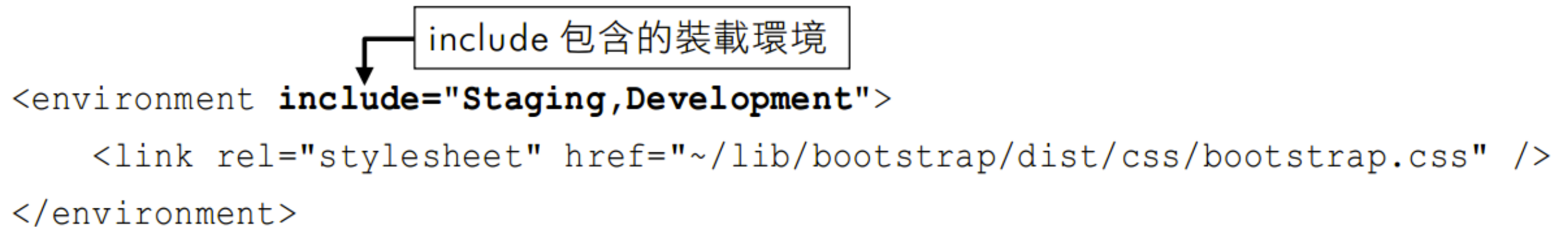
10-4-14

<environment>

- 現有裝載（Hosting）環境的不同，而有條件轉譯輸出其所包含的內容



```
<environment names="Staging,Development">  
  <link rel="stylesheet" href="/lib/bootstrap/dist/css/bootstrap.css" />  
</environment>
```



```
<environment include="Staging,Development">  
  <link rel="stylesheet" href="/lib/bootstrap/dist/css/bootstrap.css" />  
</environment>
```


exclude 排除的裝載環境



```
<environment exclude="Development">  
  <link rel="stylesheet"  
    href="https://stackpath.bootstrapcdn.com/bootstrap/4.3.1/css/bootstrap.min.css"  
    asp-fallback-href="~/lib/bootstrap/dist/css/bootstrap.min.css"  
    asp-fallback-test-class="sr-only" asp-fallback-test-property="position"  
    asp-fallback-test-value="absolute"  
    crossorigin="anonymous"  
    integrity="sha384-ggOyR0iXCbMQv3Xipma34MD+dH/1fQ784/j6cY/iJTQUOhcWr7x9JvoRxT2MZw1T"/>  
</environment>
```


Tag Helpers

加入、移除和範圍管理

10-5

- 預設 Tag Helpers 會套用到專案所有 Views，讓 Views 皆能使用標籤協助程式，是因為_ViewImports.cshtml 中有一行設定

```
@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers
```

- Tag Helpers 也能做加入、移除和退出的調整
- **@addTagHelper**：加入標籤協助程式
- **@removeTagHelper**：移除標籤協助程式
- **!**退出字元：個別 element 退出標籤協助程式

使用@addTagHelper 加入標籤協助程式

10-5-1

何時會用到@addTagHelper 加入標籤協助程式？

- 加入自訂的標籤協助程式
- 限定View或View資料夾加入內建或自訂標籤協助程式
- 例如 CustomTagHelpers 專案自訂了 Tag Helpers，在 _ViewImports.cshtml 引用的語法

 Views/_ViewImports.cshtml

```
@addTagHelper *, CustomTagHelpers
```

← 自訂組件名稱

← *星號使用組件中所有的 Tag Helpers

引用自訂組件中的 Email Tag Helper

```
@addTagHelper CustomTagHelpers.TagHelpers.EmailTagHelper, CustomTagHelpers
@addTagHelper CustomTagHelpers.TagHelpers.E*, CustomTagHelpers
@addTagHelper CustomTagHelpers.TagHelpers.Email*, CustomTagHelpers
```


使用@removeTagHelper 移除標籤協助程式

10-5-2

- ✦ 個別 View 移除內建的 Anchor Tag Helper (在檢視.cshtml 中移除)

```
@removeTagHelper Microsoft.AspNetCore.Mvc.TagHelpers.AnchorTagHelper,
    Microsoft.AspNetCore.Mvc.TagHelpers
```

- ✦ 個別 View 移除自訂 EmailTagHelper (在檢視.cshtml 中移除)

```
@removeTagHelper CustomTagHelpers.TagHelpers.EmailTagHelper, CustomTagHelpers
```

- ✦ 個別 View 移除全部內建的 Tag Helpers (在檢視.cshtml 中移除)

```
@removeTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers
```

- ✦ 針對所有 Views 作移除，是在_ViewImports.cshtml 中宣告

 Views/_ViewImports.cshtml

```
@removeTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers
```

- ✦ 若要針對 Views 下某個控制器對映的資料夾作移除，例如在 Home 控制器的檢視資料夾加入_ViewImports.cshtml，並設定

 Views/Home/_ViewImports.cshtml

```
@removeTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers
```


單一個別 elements 退出標籤協助程式

10-5-3

退出標籤協助程式

- 若想針對單一 element 退出移出標籤協助程式，可在 element 前後使用！符號
- 如此該 element 上標籤協助程式將會失去作用，回歸成原始的 HTML elements
- 同時！符號會讓 HTML elements 不再出現 asp-xxx 的 IntelliSense 提示

退出標籤協助程式語法

anchor標籤協助程式

```
<a asp-controller="Home" asp-action="Index">Home/Index</a>
```

HTML輸出帶href

```
<a href="/">Home/Index</a>
```

用！退出

```
<!a asp-controller="Home" asp-action="Index">Home/Index</!a>
```

HTML輸出沒有href屬性

```
<a asp-controller="Home" asp-action="Index">Home/Index</a>
```

用！退出

用_ViewImports.cshtml 控制Tag Helpers套用範圍

10-5-4

Tag Helpers 套用範圍

以@addTagHelper來說有三種範圍設定方式：

- Views/_ViewImports.cshtml 中宣告@addTagHelper (對 Views 資料夾下所有檢視作用)
- Views/{controller}/_ViewImports.cshtml 宣告@addTagHelper (控制器資料夾下所有檢視作用)
- 在個別 View 直接宣告@addTagHelper (只對該 View 檢視有影響)

以@tagHelperPrefix
明確啟用Tag Helpers

10-5-5

明確啟用 Tag Helpers

- 若不想讓 HTML elements 自動啟用 Tag Helpers，直到明確使用自訂的前綴字串，才啟用 Tag Helpers 支援，可用@tagHelperPrefix 設定
- 方式是在_ViewImports.cshtml 加入「@tagHelperPrefix tag:」宣告



Views/_ViewImports.cshtml

```
@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers
```

```
@tagHelperPrefix tag: ← @tagHelperPrefix
```

```
<tag:a asp-controller="Home" asp-action="Index">Index</tag:a>
```



Tag:自訂的前綴字串

自訂標籤協助程式

10-6

自訂標籤協助程式

自訂 Tag Helpers 步驟：

- ① 新增自訂類別，並繼承 TagHelper 類別，實作 Process 主要方法
- ② 在 Views/_ViewImports.html 加入自訂 Tag Helper 類別參考
- ③ 在 View 檢視使用自訂 Tag Helper 的 Tag element

自訂的 EmailTagHelper.cs 類別

繼承 TagHelper 類別

```
public class EmailTagHelper : TagHelper
{
    public const string DomainName = "codemagic.com.tw";
    public string MailTo { get; set; }
```

實作 Process 方法

```
public override void Process(TagHelperContext context, TagHelperOutput output)
{
    //輸出element Tag名稱
    output.TagName = "a";

    //設定Email Address
    var mailAddress = $"{MailTo}@{DomainName}";

    //設定href屬性
    output.Attributes.SetAttribute("href", $"mailto:{mailAddress}");

    //設定element內容文字
    output.Content.SetContent(mailAddress);
}
```


在_ViewImports.cshtml 加入自訂標籤協助程式參考

```
@addTagHelper *, CoreMvc7_TagHelpers
```

在 Email 檢視中使用自訂的<email>標籤協助程式

```
<email mail-to="services"></email>
```

HTML輸出

```
<a href="mailto:services@codemagic.com.tw">services@codemagic.com.tw</a>
```

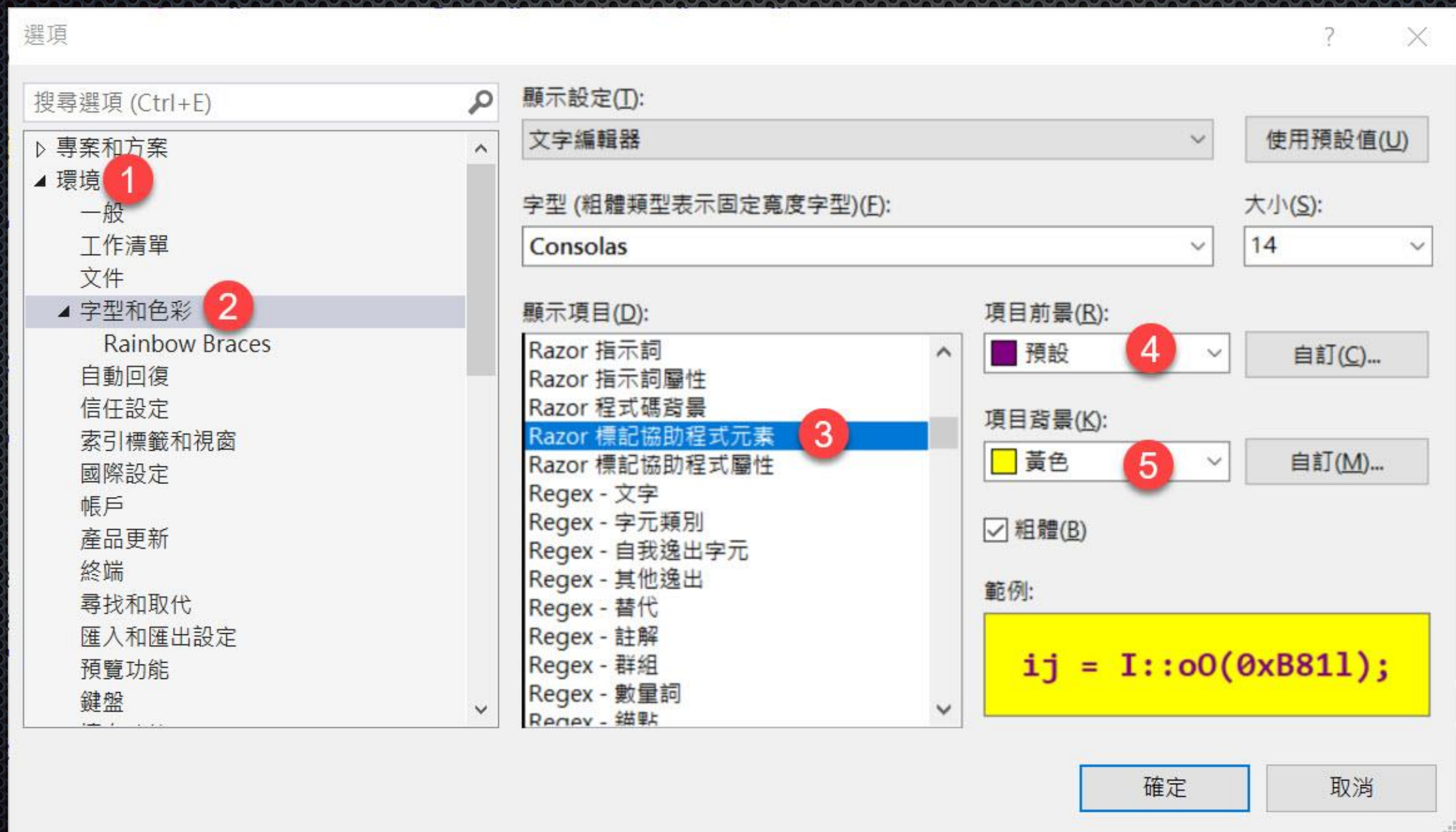

範例 10-13 建立自訂非同步 Email 標籤協助程式

自訂標籤協助程式字型與色彩

10-7

自訂 Tag Helper element 字型與顏色

- 在【工具】→【選項】→【環境】→【字型與色彩】→顯示項目的【Razor標記協助程式元素】和【Razor標記協助程式屬性】做調整



The End

奚江華個人經歷簡介

- 著有ASP.NET 2.0、3.5、4.5、4.6等熱門暢銷書籍，以及**ASP.NET MVC 5.x範例完美演繹**、**ASP.NET Core 3.x MVC跨平台範例實戰演練**
- 歷任台灣微軟MSDN、TechEd、TechDay研討會講師
- 曾數屆當選微軟MVP
- 國內各大上市公司及大專院校之培訓講師
- 曾發表文章刊於ITHOME電腦週刊

碼魔法FB

- CodeMagic碼魔法軟體學院Facebook

www.facebook.com/codemagictw